

A **class method** is a method that receives the **class** (`cls`) instead of an instance (`self`).

Simple Python Example

```
class Dog:
    species = "Canis familiaris"

    @classmethod
    def get_species(cls):
        return cls.species

print(Dog.get_species())
```

Output:

```
Canis familiaris
```

Why this is a class method

- `@classmethod` tells Python to pass the **class** as the first argument.
- `cls.species` accesses the class attribute.
- You can call it on the class (`Dog.get_species()`) or an instance.

A **class method** shines when you need behavior that depends on the **class itself**, especially when subclasses should automatically inherit or override that behavior. This is something an **instance method** *cannot* do (it only knows about one object) and a **static method** *cannot* do (it has no access to class state at all).

The one thing only a class method can do

A class method can **create new instances of the class (or subclass)** *without knowing the exact class name ahead of time*. This is impossible with:

- **Instance methods** → they already *are* an instance, so they can't create new ones.
- **Static methods** → they don't receive `cls`, so they can't construct the correct subclass.

Example: A class method that returns different subclasses

```
class Animal:
    def __init__(self, name):
        self.name = name

    @classmethod
    def from_sound(cls, sound):
        if sound == "woof":
            return Dog("Doggo")
        elif sound == "meow":
            return Cat("Kitty")
        return cls("Unknown")

class Dog(Animal):
    pass

class Cat(Animal):
    pass

a = Animal.from_sound("woof")
print(type(a), a.name)
```

Output:

```
<class '__main__.Dog'> Doggo
```

Why this *must* be a class method

- The method needs access to `cls`, because the correct class to instantiate may be a subclass.
- If you called `Dog.from_sound("woof")`, the method still works and returns a `Dog`.
- A static method cannot do this because it has **no access to** `cls`.
- An instance method cannot do this because it already represents *one specific object*, not the class.

This lets you create a `User` from JSON **without hard-coding the class name**, which is essential if `User` is subclassed.

An instance method

Below is the **broken version**. Notice: `create()` is now an **instance method**, so it receives `self` instead of `cls`.

```

class Animal:
    def create(self, kind):
        if kind == "dog":
            return Dog()
        if kind == "cat":
            return Cat()
        return Animal()

class Dog(Animal):
    pass

class Cat(Animal):
    pass

# Trying to use it the same way:
a = Animal.create("dog")    # <-- This will FAIL

```

✗ What happens?

You'll get:

```
TypeError: Animal.create() missing 1 required positional argument: 'kind'
```

Why? Because **instance methods require an instance**, so Python tries to call:

```
Animal.create(Animal, "dog")
```

But you didn't create an instance first.

✓ What you *must* do with an instance method

To make it work, you'd have to write:

```

animal = Animal()
a = animal.create("dog")

```

But now the logic is **nonsensical**:

- You need an *existing* animal to create a *new* animal
- Subclass polymorphism breaks: calling `Dog().create("dog")` still returns a new `Dog()`, but only because you hard-coded it
- You cannot use `cls` to automatically pick the correct subclass
- You cannot use it as a factory anymore

Why this fails (the core reason)

A method must know **which class to instantiate**, and only a **class method** receives `cls`.

An **instance method** receives `self`, which is a *single object*, not the class. It cannot:

- Create new instances of its own class
- Create instances of subclasses
- Behave polymorphically
- Be called on the class itself